

COSC 1100



Week 9 - Functions

Source: unless otherwise noted, all material taken from Programming Fundamentals A Modular Structured Approach, 2nd Edition, KENNETH LEROY BUSBEE AND DAVE BRAUNSCHWEIG



COSC 1100

Week 9 - Functions



Modular Programming



Modular Programming

- A software design technique
 - Separates the functionality of a program into independent, interchangeable modules
 - Each module contains everything needed to execute one aspect of the functionality that's needed for the entire application
- Uses the concept of a code block (grouping lines of code into a unit)
- Originally called a sub-program
- Other names used over the years: macro, sub-routine, procedure, module, function
- The term **function** is used in most of the predominant languages today.
- Functions allow us to take large complicated programs and divide them into smaller manageable pieces

Modular Programming

Functions generally fall into 2 categories:

1. **Program Control** – unique to the program being written. Used to simply sub-divide and control the program.
2. **Specific Task** – designed so they could be used with several programs. Any program that needs to do the specific task they are designed for. AKA “building blocks”. If they are already coded and tested we can use them and more efficiently write a large program.

Modular Programming

- Main program must identify/establish the functions to be used.
- Depending on the language, there may be a formal way to:
 1. Define a function (**definition** – the actual code to be executed)
 2. **Call** a function (actually use the function to perform a task)
 3. Declare a function (a **prototype** is a declaration to a compiler)
- **Defining** and **calling** → common across all programming languages
- Declaring with **prototypes** → specific to certain languages (like C and C++)

Modular Programming - Communications

- Data can be communicated between the calling program and the function being called.
- Data communication is what allows us to build functions that can be used in many programs.
- How data is communicated and the rules for communication vary a lot in different programming languages.
- The data items that are communicated are called **parameters** (parameter passing).

Modular Programming - Communications

- 4 possibilities for parameter passing:
 1. No parameters in, no parameters out
 2. No parameters in, some parameters out
 3. Some parameters in, some parameters out
 4. Some parameters in, no parameters out

Modular Programming – Program Control Function

- Special function called “main” in many languages
- This is where the program starts executing code and usually where it stops.
- Optional in Python

Modular Programming – Specific Task Function

- General layout of a function in Python (dynamically typed language):

```
def <function identifier name>(<identifier name for input value>):  
    # lines of code  
    return <value>    # optional!
```

- Remember some languages use braces to identify a code block—Python uses indentation.

Modular Programming – Program Layout

- We now have several items to place at the top of our code file! Here they are in order:
 1. Documentation – start with your header comments
 2. “import” statements – used to access standard library functions
 3. Language-specific code such as namespace references or function prototypes – we aren’t using these, so no worries!
 4. Global or module-level constants and variables – our declarations.
 5. Function definitions – they go after the declarations

COSC 1100

Week 9 - Functions

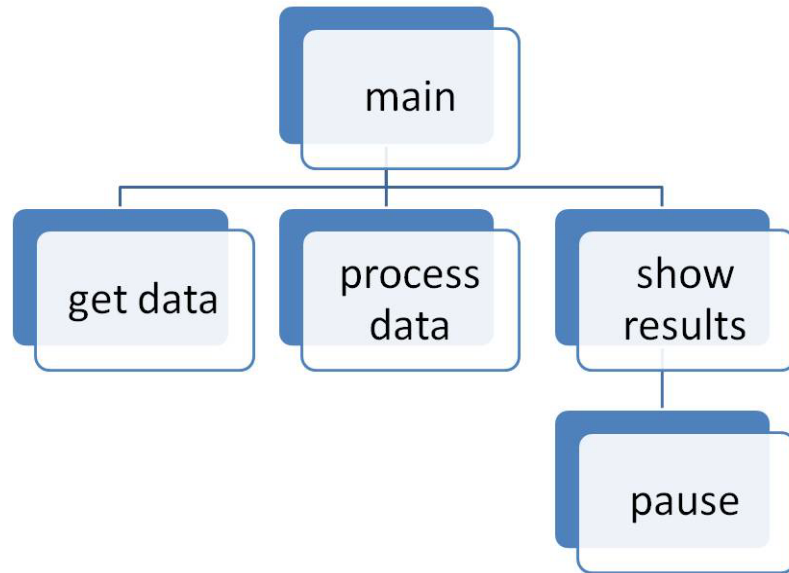


Hierarchy or Structure Chart



Hierarchy or Structure Chart

- Hierarchy chart (aka structure chart) show relationship between the program modules.
- Like a business organization chart:



- The modules (or functions) above are calling the functions on the next level down.
- “Big picture” diagram

COSC 1100

Week 9 – Functions



Function Examples



Function Examples

An example application – Temperature Conversion Program

- Each function will act as a miniature program, with its own input, processing and output

Function Name	Purpose	Parameters (input)	Return Value (output)
Main	Main control program	None	None
GetFahrenheit	Input	None	Fahrenheit value
CalculateCelcius	Processing	Fahrenheit value	Celsius value
DisplayResults	Output	Fahrenheit, celsius	None

Function Example – Pseudocode for the Main function

Function Main

... This program asks the user for a Fahrenheit temperature,
... converts the given temperature to Celsius,
... and displays the results.

Declare Real fahrenheit

Declare Real celsius

Assign fahrenheit = GetFahrenheit()

Assign celsius = CalculateCelsius(fahrenheit)

Call DisplayResult(fahrenheit, celsius)

End

Function Example – Other Function Pseudocode

```
Function GetFahrenheit
    Declare Real fahrenheit_out
    Output "Enter Fahrenheit temperature:"
    Input fahrenheit_out
    Return Real fahrenheit_out

Function CalculateCelsius (Real fahrenheit_in)
    Declare Real celsius_out
    Assign celsius_out = (fahrenheit_in - 32) * 5 / 9
    Return Real celsius_out

Function DisplayResult (Real fahrenheit_in, Real celsius_in)
    Output fahrenheit_in & "° Fahrenheit is " & celsius_in & "° Celsius"
End
```


Function Example – Desk Check

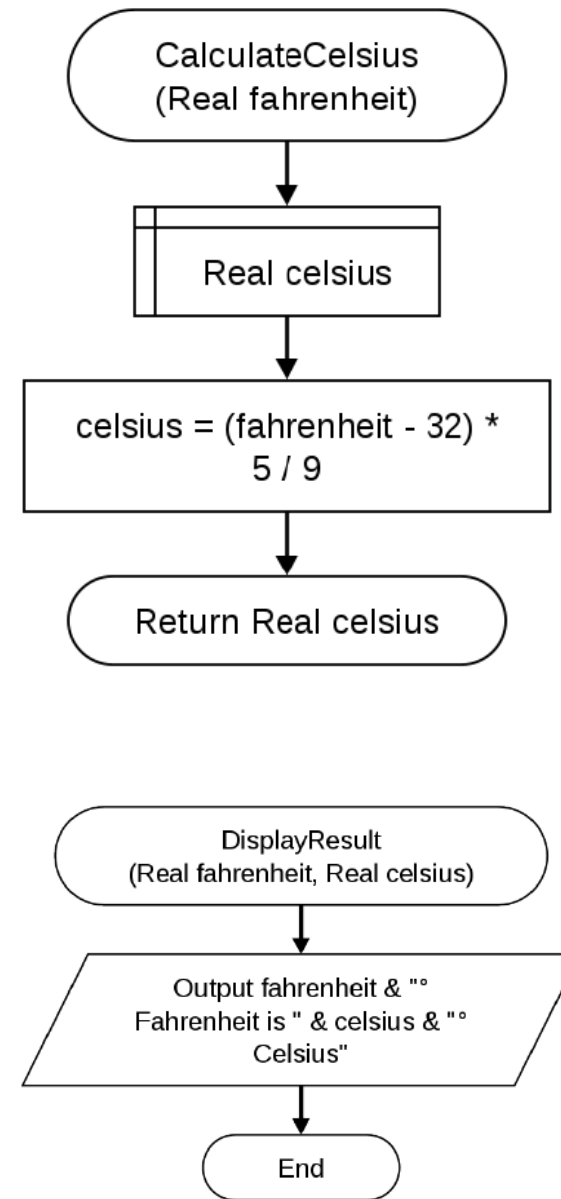
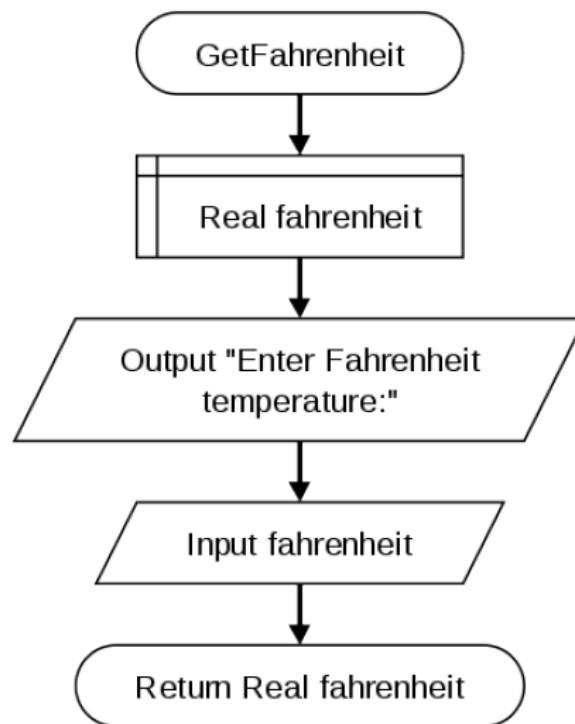
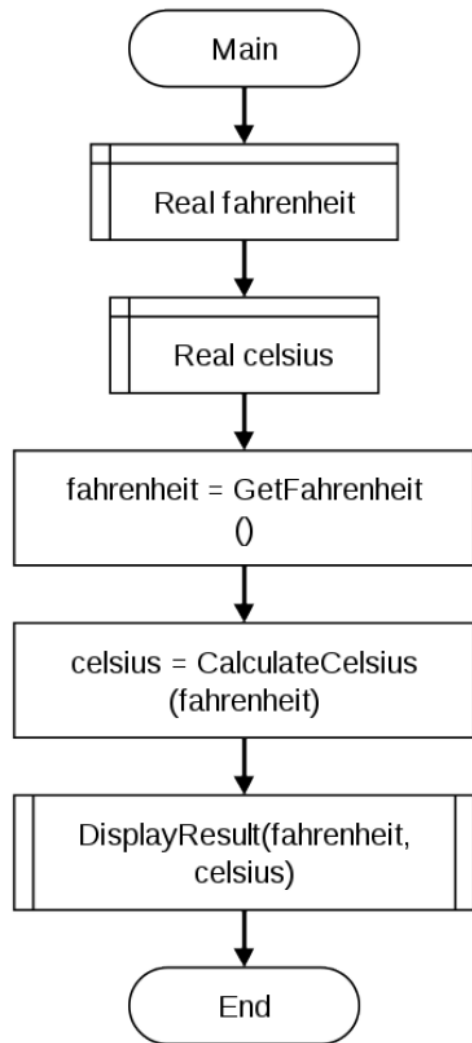
Example Input and Output:

```
Enter Fahrenheit temperature:
```

```
100
```

```
100° Fahrenheit is 37.7777777777778° Celsius
```

Function Example – Flowchart



COSC 1100

Week 9 – Functions



Parameters and Arguments



Parameters and Arguments

- **Parameter** – special kind of variable used in a function. Refers to one of the inputs to the function.
- **Argument** – a value provided as an input when the function is called. The value is assigned to one of the function's parameters.

Parameters and Arguments

Parameter

```
Function CalculateCelsius (Real fahrenheit_in)
    Declare Real celsius_out
    Assign celsius_out = (fahrenheit_in - 32) * 5 / 9
Return Real celsius_out
```

```
Function main
    ...
    Assign celsius = CalculateCelsius(100)
    ...
End
```

Argument

Parameters and Arguments

- What about functions with 2 parameters?

```
Function DisplayResult (Real fahrenheit_in, Real celsius_in)
    Output fahrenheit_in & "° Fahrenheit is " & celsius_in & "° Celsius"
End

Function main
    ...
    DisplayResult(98.6, 37.0)
    ...
End
```

Would display:

```
98.6° Fahrenheit is 37.0° Celsius
```

Parameters and Arguments

- What if you did this?

```
Function DisplayResult (Real fahrenheit_in, Real celsius_in)
    Output fahrenheit_in & "° Fahrenheit is " & celsius_in & "° Celsius"
End

Function main
    ...
    DisplayResult(37.0, 98.6)
    ...
End
```

Would display:

```
37.0° Fahrenheit is 98.6° Celsius
```

Which is not correct!

Parameters and Arguments

- Some programming languages, such as Python, support named parameters. If you use parameter names, then position is ignored.
- Any of these would produce valid results:

```
CalculateCelsius(98.6, 37.0)  
CalculateCelsius(fahrenheit_in=98.6, celsius_in=37.0)  
CalculateCelsius(celsius_in=37.0, fahrenheit_in=98.6)
```


COSC 1100

Week 9 – Functions



Call by Value vs. Call by Reference



Call by Value vs. Call by Reference

- **Call by Reference** – parameters passed by calling functions may be modified by called functions.
- **Call by Value** – Parameters passed by calling functions cannot be modified by called functions.
- Python example from:
<https://www.geeksforgeeks.org/is-python-call-by-reference-or-call-by-value/>

COSC 1100

Week 9 – Functions



Return Statement



Return Statement

- Causes program execution to leave the current function and resume at the point in code immediately after where the function was called.
- In many languages, including Python, return statements allow a function to specify a value to be passed back to the code that called the function.

COSC 1100

Week 9 – Functions



Scope



Scope

- The scope of an identifier name binding – such as a variable – is the region of a computer program where the binding is valid: i.e., where the name can be used to refer to the entity.
- The region is referred to as a scope block.
- In other parts of the program the name may refer to a different entity or to nothing at all.
- **Global Scope** – a variable is defined “outside of a function”. It is available to all of the functions.
- **Local Scope** – a variable defined “inside a function”. It is available only within that function.